

UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería de Industrial y Sistemas



Sistema de archivos virtual en memoria

Integrantes:

- Luna Roca, Fabrizio Santiago 20241123H
- Cuello Inche Manuel Benito
- Flores Zambora Airthon Daniel
- Gamrraa Cure Jeferson Fabian

Docente: JUAN LUIS HERENCIA GUERRA

Curso: Lenguaje de programación I (Imperativo)

Ciclo: 2026-I

ANÁLISIS Y DISEÑO

Sistema de archivos virtual en memoria (C)

1. Requerimientos del Sistema de Archivos (FS)

Requerimientos funcionales

El sistema debe permitir las funciones:

Crear archivos (`touch`)

El sistema debe permitir la creación de archivos dentro de los directorios, cada uno dentro tendrá un nombre propio, cada archivo nuevo será representado como nodo en la estructura del árbol

- Inicialmente el archivo debe estar vacío y ser tamaño cero
- El sistema debe validar que no exista un archivo con el mismo nombre en la carpeta, de lo contrario no se podrá crear el archivo

Crear directorios (`mkdir`)

El sistema debe permitir la creación de carpetas dentro de los directorios, los cuales contendrán nuevos archivos o subdirectorios

- Cada directorio será representado como un nodo que puede tener múltiples hijos
- No debe existir en el mismo nivel del árbol nodo con el mismo nombre, caso contrario no se creará una carpeta

Listar contenido (`ls`)

Se mostrará el nombre de todos los directorios y archivos contenido de un directorio determinado

- Se muestra todo los directorios y archivos dentro del nodo que determinamos pero por defecto se accedera al nodo en el que nos encontremos
- No se modifica nada de la estructura interna

Escribir en archivos (`write`)

Nos permite escribir o modificar archivos ya existentes

- Si tenemos un archivo podemos sobrescribir o añadir lo que deseemos
- No se debe permitir hacer esto sobre un tipo directorio (carpeta)

Leer archivos (`cat`)

Muestra todo lo que esta escrito dentro de un archivo

- Se debe mostrar el contenido creado en el archivo

- En caso no se pueda leer o no exista, se mostrará un mensaje de error
- No se permite leer un directorio de lo contrario saldrá un mensaje de erro

Eliminar archivos/directorios (rm)

El sistema debe permitir eliminar archivos y directorios.

- Al momento de eliminar archivos, se debe borrar el contenido y el nodo asociado a los datos
- En caso de los directorios se usará una función recursiva que elimina directorios y archivos dentro para liberar memoria
- Asegurarse que se afectó a la estructura del árbol al terminar

Navegar entre rutas (cd)

El sistema debe permitir desplazarse a través de la estructura de directorios, simulando el comportamiento de sistemas reales.

- Se debe manejar un directorio actual de trabajo.
- Se debe poder acceder a subdirectorios y volver al directorio padre

Requerimientos no funcionales

- **Uso eficiente de memoria dinámica (malloc, realloc, free)**

La memoria debe gestionarse usando las mismas funciones del lenguaje, usando malloc, realloc o free

- La creación de nodo es de forma dinámica
- La memoria asignada debe considerarse la cantidad de datos
- Verificar si un nodo es eliminado para no desperdiciar memoria

- **Estructura tipo árbol (similar a Linux)**

El sistema debe mantener una organización basada en una estructura de tipo árbol, asegurando que se conserve las ramas en todo momento.

- Cada nodo debe tener correctamente definido su nodo padre y en caso de ser un directorio, sus hijos.
- Debemos verificar que todos los directorios existan y referencias exactas, ni tampoco ciclos
- Cualquier función o operación no debe modificar la estructura, solo trabajar con lo ya hecho

- **Manejo de errores (archivo no existe, ruta inválida, etc.)**

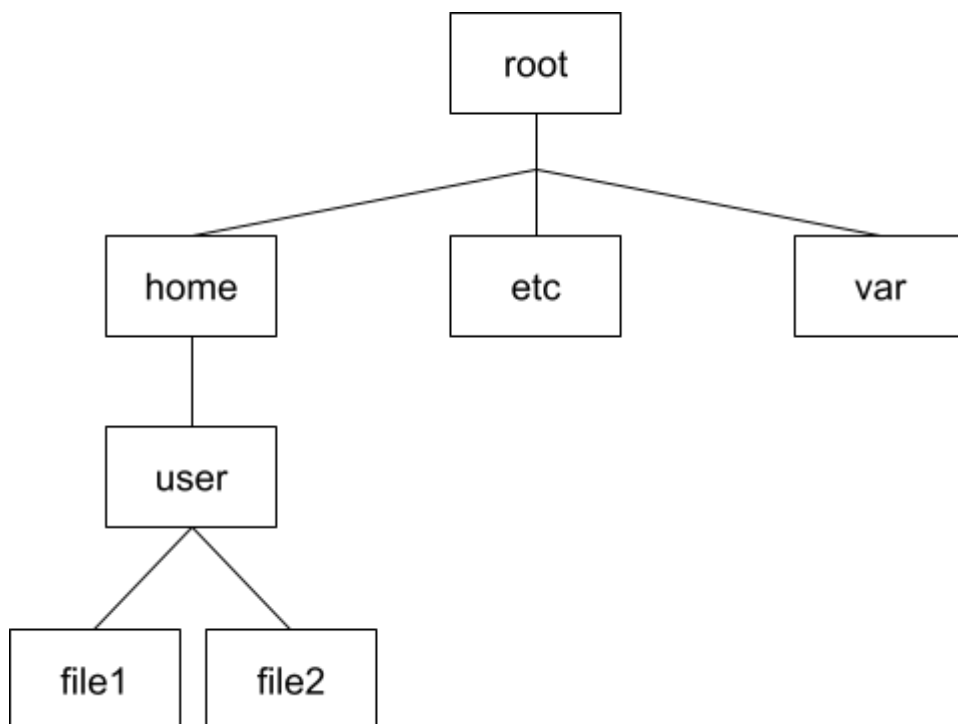
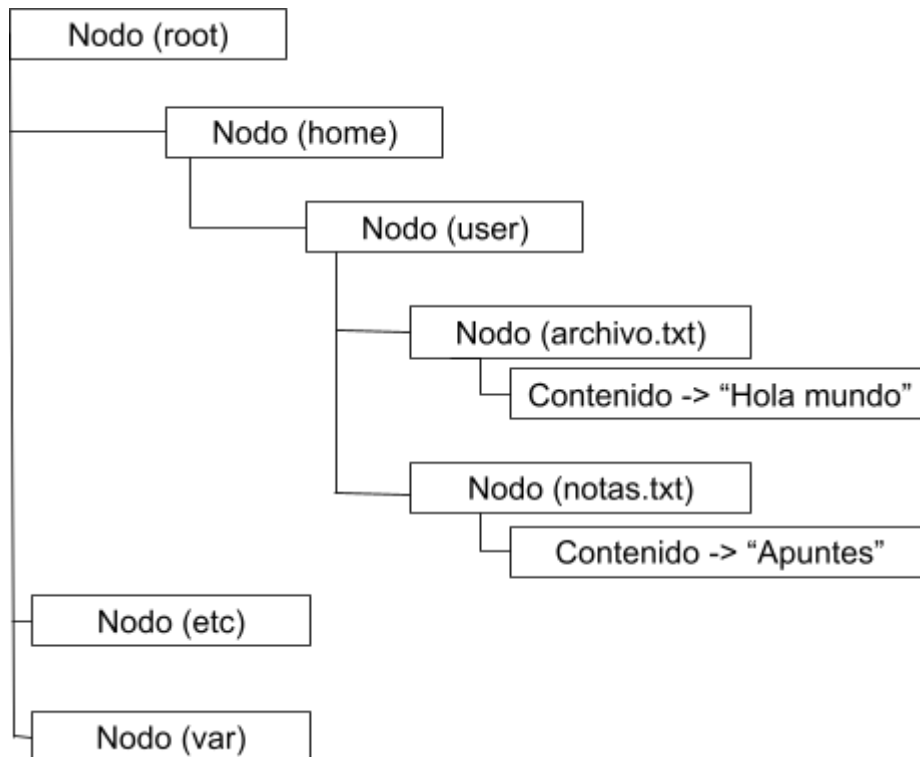
El sistema debe ser capaz de identificar errores comunes y mostrarlos en pantalla

- En caso de errores, el sistema debe mostrar mensajes claros
- Se debe validar las entradas del usuario al ejecutar y detenerse en caso no ser válida

- Ante errores controlados no se debe interrumpir dicha ejecución

2. Diseño del Árbol de Nodos

El sistema se basa en una estructura jerárquica tipo árbol:



3. Pseudocódigo de Funciones Principales

INICIO

DEFINIR **TipoNodo** = {ARCHIVO, DIRECTORIO}

ESTRUCTURA **Nodo**:

nombre
tipo
padre
hijos (arreglo dinámico de **Nodo**)
numHijos
contenido
size

FIN ESTRUCTURA

//AQUI ES LA FUNCION CREAR NODO

FUNCION **crearNodo**(nombre, tipo):

nodo ← nuevo **Nodo**
nodo.nombre ← nombre
nodo.tipo ← tipo
nodo.padre ← NULL
nodo.hijos ← NULL
nodo.numHijos ← 0

SI tipo == ARCHIVO ENTONCES

nodo.contenido ← NULL
nodo.size ← 0

FIN SI

RETORNAR nodo

FIN FUNCION

//FUNCION AGREGAR HIJO(ES EL ARCHIVO O CARPETA CREADO DENTRO DE UN PADRE

FUNCION **agregarHijo**(padre, hijo):

padre.numHijos ← padre.numHijos + 1
redimensionar padre.hijos

padre.hijos[padre.numHijos - 1] ← hijo
hijo.padre ← padre

FIN FUNCION

FUNCION **buscarHijo**(padre, nombre):

PARA i ← 0 HASTA padre.numHijos - 1 HACER
SI padre.hijos[i].nombre == nombre ENTONCES
RETORNAR padre.hijos[i]

FIN SI

FIN PARA
RETORNAR NULL
FIN FUNCION

//LA FUNCION resolverRuta SIRVE PARA ENCONTRAR UN NODO DENTRO DEL ARBOL USANDO UNA RUTA TIPO LINUX.

FUNCION resolverRuta(ruta, actual, root):
 SI ruta empieza con "/" **ENTONCES**
 nodoActual ← root
 SINO
 nodoActual ← actual
 FIN SI

 partes ← dividir ruta por "/"

 PARA CADA parte **EN** partes **HACER**
 SI parte == "" O parte == "." **ENTONCES**
 CONTINUAR
 FIN SI

 SI parte == ".." **ENTONCES**
 nodoActual ← nodoActual.padre
 SINO
 nodoActual ← buscarHijo(nodoActual, parte)
 FIN SI

 SI nodoActual == NULL **ENTONCES**
 RETORNAR NULL
 FIN SI
 FIN PARA

 RETORNAR nodoActual
FIN FUNCION

// LA FUNCION separarRuta SEPARA EL NOMBRE DEL HIJO DE SU DIRECCION EN EL SISTEMA DE ARCHIVOS

FUNCION separarRuta(ruta):
 encontrar última "/"
 padreRuta ← parte antes de "/"
 nombre ← parte después de "/"
 RETORNAR (padreRuta, nombre)
FIN FUNCION

//FUNCION mkdir SIRVE PARA CREAR UN DIRECTORIO (carpeta) DENTRO DEL SISTEMA DE ARCHIVOS

FUNCION mkdir(ruta, actual, root):

```
(padreRuta, nombre) ← separarRuta(ruta)
padre ← resolverRuta(padreRuta, actual, root)
```

```
SI padre == NULL ENTONCES
    IMPRIMIR "Error: ruta inválida"
    RETORNAR
FIN SI
```

```
SI buscarHijo(padre, nombre) != NULL ENTONCES
    IMPRIMIR "Ya existe"
    RETORNAR
FIN SI
```

```
nuevo ← crearNodo(nombre, DIRECTORIO)
agregarHijo(padre, nuevo)
FIN FUNCION
```

//FUNCION touch CREAR UN NUEVO ARCHIVO VACÍO DENTRO DE UN DIRECTORIO

```
FUNCION touch(ruta, actual, root):
    (padreRuta, nombre) ← separarRuta(ruta)
    padre ← resolverRuta(padreRuta, actual, root)
```

```
SI padre == NULL ENTONCES
    IMPRIMIR "Error"
    RETORNAR
FIN SI
```

```
nuevo ← crearNodo(nombre, ARCHIVO)
agregarHijo(padre, nuevo)
FIN FUNCION
```

//FUNCION ls ES PARA MOSTRAR LA LISTA DE ARCHIVOS Y DIRECTORIOS QUE EXISTEN DENTRO DE UNA CARPETA

```
FUNCION ls(ruta, actual, root):
    nodo ← resolverRuta(ruta, actual, root)
```

```
SI nodo == NULL ENTONCES
    IMPRIMIR "No existe"
    RETORNAR
FIN SI
```

```
PARA i ← 0 HASTA nodo.numHijos - 1 HACER
    IMPRIMIR nodo.hijos[i].nombre
FIN PARA
FIN FUNCION
```

// FUNCION write SIRVE PARA ESCRIBIR O GUARDAR TEXTO DENTRO DE UN ARCHIVO

FUNCION write(ruta, texto, actual, root):

nodo ← resolverRuta(ruta, actual, root)

SI nodo == NULL O nodo.tipo != ARCHIVO ENTONCES

IMPRIMIR "Error"

RETORNAR

FIN SI

liberar nodo.contenido

nodo.contenido ← copiar(texto)

nodo.size ← longitud(texto)

FIN FUNCION

//FUNCION cat PARA LEER Y MOSTRAR EL CONTENIDO DE UN ARCHIVO

FUNCION cat(ruta, actual, root):

nodo ← resolverRuta(ruta, actual, root)

SI nodo == NULL O nodo.tipo != ARCHIVO ENTONCES

IMPRIMIR "Error"

RETORNAR

FIN SI

IMPRIMIR nodo.contenido

FIN FUNCION

//FUNCION eliminarRecursoivo BORRAR UN DIRECTORIO JUNTO CON TODO SU CONTENIDO (SUBCARPETAS Y ARCHIVOS)

FUNCION eliminarRecursoivo(nodo):

SI nodo.tipo == DIRECTORIO ENTONCES

PARA i ← 0 HASTA nodo.numHijos - 1 HACER

eliminarRecursoivo(nodo.hijos[i])

FIN PARA

FIN SI

liberar nodo.contenido

liberar nodo.hijos

liberar nodo

FIN FUNCION

//FUNCION rm ES PARA ELIMINAR UN ARCHIVO O DIRECTORIO DEL SISTEMA

FUNCION rm(ruta, actual, root):

nodo ← resolverRuta(ruta, actual, root)


```
SI nodo == NULL ENTONCES
    IMPRIMIR "No existe"
    RETORNAR
FIN SI
```

```
padre ← nodo.padre
```

```
eliminarRecurso(nodo)
```

```
quitar nodo de padre.hijos
padre.numHijos ← padre.numHijos - 1
```

FIN FUNCION

//FUNCIÓN cd SE ENCARGA DE CAMBIAR EL DIRECTORIO ACTUAL PARA NAVEGAR DENTRO DEL SISTEMA DE ARCHIVOS

```
FUNCION cd(ruta, actual, root):
    nodo ← resolverRuta(ruta, actual, root)

    SI nodo == NULL O nodo.tipo != DIRECTORIO ENTONCES
        IMPRIMIR "Error"
        RETORNAR actual
    FIN SI
```

```
RETORNAR nodo
```

FIN FUNCION

PROGRAMA PRINCIPAL:

```
root ← crearNodo("/", DIRECTORIO)
actual ← root
```

MIENTRAS VERDADERO HACER
LEER comando

SEGÚN comando HACER:

```
"mkdir" → llamar mkdir
"touch" → llamar touch
"ls" → llamar ls
"write" → llamar write
"cat" → llamar cat
"rm" → llamar rm
"cd" → actual ← cd(...)
"exit" → TERMINAR
```

FIN SEGÚN

FIN MIENTRAS

FIN

4. Tabla de Comandos y Funciones

Comando	Función en C	Descripción
mkdir	crearDirectorio()	Crea un directorio
touch	crearArchivo()	Crea un archivo vacío
ls	listar()	Lista contenido
cat	leerArchivo()	Muestra contenido
write	escribirArchivo()	Escribe contenido
rm	eliminar()	Borrar archivo o directorio
cd	cambiarDirectorio()	Navega entre rutas